



Jogo Web de Gestao de Recursos

Projeto Final - Desenvolvimento de Aplicacoes Web
Escola Superior de Ciencias Empresariais - IPS
Ano Letivo 2025/2026

Bernardo Pinheiro, Ines Medeiros, Ricardo Janz

1. Introdução

O presente relatório descreve o projeto final da unidade curricular Desenvolvimento de Aplicações Web (DAW), lecionada na Escola Superior de Ciências Empresariais do Instituto Politécnico de Setúbal (IPS/ESCE) no ano letivo 2025/2026.

O projeto consiste no desenvolvimento de um jogo web de gestão de recursos, com temática inspirada no universo Minecraft. O jogador controla a personagem Steve num cenário 2D, onde pode cortar árvores para obter madeira, minerar pedras, construir edifícios (Mesa de Trabalho, Fornalha e Mineradora de Diamantes), iniciar tarefas temporizadas e recolher recompensas. O jogo inclui ainda um sistema de evolução do machado, uma loja de skins e um leaderboard global.

O objetivo principal foi consolidar os conhecimentos adquiridos ao longo das aulas práticas (laboratórios 1 a 8), nomeadamente nos domínios da programação JavaScript (manipulação do DOM, estruturas de dados, persistência), programação servidor em Python/Flask (padrão MVC, autenticação, base de dados) e integração frontend-backend através da Fetch API.

2. Arquitetura da Solução

2.1 Visão Geral

A aplicação segue o padrão MVC (Model-View-Controller), com uma separação clara entre a lógica de negócio, a apresentação e os dados. O backend é implementado em Flask (Python) e o frontend em HTML5, CSS3 e JavaScript vanilla, sem dependências de frameworks JavaScript externas.

A comunicação entre frontend e backend é feita através de requisições HTTP assíncronas utilizando a Fetch API, com dados em formato JSON. As páginas HTML são renderizadas no servidor usando o motor de templates Jinja2.

2.2 Estrutura de Diretórios

```
minecraft2d/
├── server.py           # Ponto de entrada
├── auth.py             # Blueprint autenticação
├── game.py             # Blueprint do jogo
├── settings.py         # Configurações
├── extensions.py       # LoginManager
├── requirements.txt    # Dependências
├── base_dados.sql      # Esquema SQL
├── api_documentacao.md # Documentação API
├── README.md           # Instruções
├── models/
│   ├── __init__.py    # Modelos + user_loader
│   ├── database.py    # Database + modelos
│   └── minecraft2d.sqlite # Base de dados
├── templates/         # 6 templates Jinja2
├── static/
│   ├── css/           # 10 folhas modulares
│   ├── js/            # game.js, theme.js
│   └── img/           # Recursos gráficos
```

2.3 Tecnologias Utilizadas

- Backend: Flask, Flask-Login, SQLite 3, passlib (pbkdf2_sha256)
- Frontend: HTML5 semântico, CSS3 (Flexbox, Grid, custom properties), JavaScript vanilla
- Autenticação: Sessões baseadas em cookies com Flask-Login
- Base de Dados: SQLite 3 com row factory (sqlite3.Row)

- Controlo de Versao: Git + GitHub

3. Base de Dados

3.1 Esquema Relacional

A base de dados é composta por 6 tabelas, implementadas em SQLite 3. O esquema completo encontra-se no ficheiro `base_dados.sql`.

Tabela `users` - Autenticacao e recursos do jogador:

<code>id</code>	<code>INTEGER PRIMARY KEY AUTOINCREMENT</code>
<code>username</code>	<code>VARCHAR(80) NOT NULL UNIQUE</code>
<code>email</code>	<code>VARCHAR(120) NOT NULL UNIQUE</code>
<code>password_hash</code>	<code>VARCHAR(255) NOT NULL</code>
<code>wood</code>	<code>INTEGER DEFAULT 26</code>
<code>stone</code>	<code>INTEGER DEFAULT 26</code>
<code>iron</code>	<code>INTEGER DEFAULT 0</code>
<code>diamonds</code>	<code>INTEGER DEFAULT 0</code>
<code>has_axe</code>	<code>INTEGER DEFAULT 0</code>
<code>axe_level</code>	<code>INTEGER DEFAULT 0</code>
<code>skin</code>	<code>TEXT DEFAULT 'default'</code>
<code>created_at</code>	<code>TEXT NOT NULL</code>

Tabela `building_slots` - Slots de construcao (4 por jogador):

<code>id</code>	<code>INTEGER PK AUTOINCREMENT</code>
<code>user_id</code>	<code>INTEGER NOT NULL FK -> users.id</code>
<code>slot_number</code>	<code>INTEGER NOT NULL (1-4)</code>
<code>building_type</code>	<code>VARCHAR(40)</code>
<code>state</code>	<code>VARCHAR(20) DEFAULT 'empty'</code>
<code>action_type</code>	<code>VARCHAR(40)</code>
<code>started_at</code>	<code>TEXT</code>
<code>ready_at</code>	<code>TEXT</code>
<code>created_at</code>	<code>TEXT NOT NULL</code>
<code>UNIQUE(user_id, slot_number)</code>	

Tabelas `trees` e `stones` - Recursos do mapa com cooldown:

<code>id</code>	<code>INTEGER PK AUTOINCREMENT</code>
<code>column</code>	<code>INTEGER NOT NULL UNIQUE</code>
<code>chopped_at</code>	<code>TEXT / mined_at TEXT</code>
<code>removed_at</code>	<code>TEXT</code>
<code>created_at</code>	<code>TEXT NOT NULL</code>

Tabela `buildings` - Catalogo estatico de construcoes:

<code>key</code>	<code>VARCHAR(40) PK</code>
<code>name</code>	<code>VARCHAR(80)</code>
<code>cost_wood/stone/iron</code>	<code>INTEGER</code>
<code>construction_seconds</code>	<code>INTEGER</code>
<code>task_name</code>	<code>VARCHAR(80)</code>
<code>task_seconds</code>	<code>INTEGER</code>
<code>reward_wood/stone/iron/diamond</code>	<code>INTEGER</code>
<code>description</code>	<code>VARCHAR(255)</code>

3.2 Dados Iniciais (Seed)

A base de dados é inicializada com 3 arvores (colunas 2, 8, 15), 4 pedras (colunas 1, 4, 7, 10) e 3 tipos de construcao:

- Mesa de Trabalho: custo 15 madeira + 5 pedra, 20s construcao, fabrica/upgrade do machado
- Fornalha: custo 10 madeira + 15 pedra, 25s construcao, produz 1 ferro por tarefa
- Mineradora de Diamantes: custo 10 madeira + 10 pedra + 20 ferro, 30s, produz 1 diamante

4. Backend (Flask)

4.1 Autenticacao

O modulo auth.py implementa as rotas de autenticacao: registo (/register), login (/login) e logout (/logout). Utiliza Flask-Login para gestao de sessoes e passlib (pbkdf2_sha256) para hashing de passwords. As validacoes incluem: campos obrigatorios, comprimento minimo da password (4 caracteres) e unicidade de username/email.

O registo cria automaticamente 4 slots de construcao vazios para o novo utilizador e regista a primeira entrada no historico de acoes.

4.2 Rotas do Jogo

O modulo game.py (Blueprint game) implementa as seguintes rotas:

- GET / | /dashboard - Pagina principal do jogo (renderizada com Jinja2)
- GET /api/state - Estado completo do jogo em JSON (user, slots, logs, arvores, pedras)
- POST /api/build/<id> - Inicia construcao num slot vazio
- POST /api/task/<id>/start - Inicia tarefa num slot pronto
- POST /api/task/<id>/collect - Recolhe recompensa de tarefa concluida
- POST /api/chop - Corta arvore (recebe coluna no JSON body)
- POST /api/mine-stone - Mina pedra (recebe coluna no JSON body)
- POST /api/slot/<id>/remove - Remove construcao de um slot
- POST /api/inventory/remove - Remove recursos do inventario manualmente
- GET /api/leaderboard - Top 10 jogadores (JSON)
- GET /skins, /api/skins, POST /api/skin/select - Sistema de personalizacao de skins

4.3 Maquina de Estados dos Slots

Cada slot de construcao segue o ciclo: empty -> building -> ready -> working -> collectable -> ready. Os timestamps (started_at, ready_at) sao guardados em ISO datetime e comparados com o tempo atual do servidor a cada requisicao, eliminando a necessidade de cron jobs ou processos em background.

4.4 Sistema do Machado

A Mesa de Trabalho permite fabricar e fazer upgrade do machado. O nivel 1 duplica a madeira obtida por arvore. Niveis superiores aumentam progressivamente o rendimento. Os custos de upgrade escalam com o nivel: niveis 1-2 usam madeira e pedra, nivel 3 adiciona ferro, niveis 4+ usam diamantes.

5. Frontend

5.1 Estrutura HTML e Templates

O frontend é composto por 6 templates Jinja2 que estendem um layout base (base.html). O layout base inclui uma barra de navegação com ligações para Dashboard, Skins, Leaderboard e Logout, além de um botão de alternância entre modo claro e escuro. As páginas de autenticação (login, register) seguem um design Minecraft temático.

5.2 CSS Modular

O estilo é organizado em 10 ficheiros CSS modulares com @import:

- variables.css: propriedades personalizadas (cores, sombras, espaçamentos)
- typography.css: font-face, body font-stack, estilos base
- layout.css: topbar, navegação, auth pages, botões, responsividade
- game.css: dashboard, cena de jogo, inventário, slots, seletor de construções
- leaderboard.css: grelha de ranking, itens, badges de podium
- skins.css: grelha de skins, cartões, botões de seleção
- theme.css: modo escuro, cena dia/noite, nuvens, estrelas, pirilampos

5.3 JavaScript (game.js)

O ficheiro game.js (884 linhas) implementa toda a lógica do lado do cliente: renderização da cena 2D (grelha, árvores, pedras, Steve), controlo de movimento (setas direcionais, teclas A/D, botões na interface), interação com recursos (clique em árvores/pedras), Fetch API para todas as requisições ao backend, atualização em tempo real do inventário e slots com indicadores de progresso, e sistema de notificações toast para feedback ao utilizador.

5.4 Responsividade

A interface é responsiva com 3 breakpoints: 980px (tablet), 720px e 430px (mobile). Utiliza Flexbox e CSS Grid para layout, garantindo que o jogo é jogável em qualquer dispositivo.

6. Mecanica do Jogo

6.1 Recursos

O jogo possui 4 recursos: madeira (wood), pedra (stone), ferro (iron) e diamantes (diamonds). Cada novo jogador começa com 26 de madeira e 26 de pedra. O limite maximo de cada recurso e 64 unidades.

6.2 Extracao de Recursos

O mapa contem 3 arvores (colunas 2, 8, 15) e 4 pedras (colunas 1, 4, 7, 10). Cada extracao rende 4 madeira ou 2 pedra, respetivamente. Apos extracao, o recurso fica indisponivel durante 10 segundos (cooldown).

Se o jogador possuir um machado, a madeira obtida e multiplicada por $(axe_level + 1)$. Por exemplo, com machado nivel 3, cada arvore rende $4 \times 4 = 16$ madeira.

6.3 Construcoes e Tarefas

O jogador dispoe de 4 slots de construcao. Existem 3 tipos de construcao:

- Mesa de Trabalho (cabana): Permite fabricar o machado inicial (custo: 15 madeira + 5 pedra) e fazer upgrades progressivos.
- Fornalha (mina): Funde minerio em lingotes de ferro. Custo: 10 madeira + 15 pedra. Cada tarefa concluida produz 1 ferro.
- Mineradora de Diamantes (forja): Extrai diamantes. Custo: 10 madeira + 10 pedra + 20 ferro. Cada tarefa concluida produz 1 diamante.

6.4 Ciclo de Vida de um Slot

1. Slot vazio (empty) -> jogador paga custo e inicia construcao
2. Construcao em progresso (building) -> espera pelo temporizador
3. Construcao pronta (ready) -> jogador pode iniciar uma tarefa
4. Tarefa em progresso (working) -> espera pelo temporizador
5. Tarefa concluida (collectable) -> jogador recolhe a recompensa
6. Slot volta ao estado ready para nova tarefa

6.5 Skins

O jogo inclui 5 skins: Steve (default), Steve Dourado (filtro sepia), Steve das Sombras (escuro), Steve Bronzeado (tons quentes) e Steve Rosa (tom roxo). As skins sao aplicadas via filtros CSS sobre a mesma imagem base, sem necessidade de recursos graficos adicionais.

6.6 Leaderboard

O leaderboard apresenta o top 10 de jogadores ordenados por pontuacao total (soma de todos os recursos). As tres primeiras posicoes exibem medalhas (ouro, prata, bronze). A lista e atualizada automaticamente a cada 5 segundos no frontend via Fetch API.

7. Documentacao da API

A API do backend expoe 13 endpoints. Todos requerem autenticacao via Flask-Login (cookie de sessao), exceto /login e /register. A documentacao completa com exemplos de requests e responses encontra-se no ficheiro api_documentacao.md.

Resumo dos endpoints:

Metodo	Rota	Descricao
GET	/login	Formulario de login
POST	/login	Autenticar utilizador
GET	/register	Formulario de registo
POST	/register	Criar nova conta
GET POST	/logout	Terminar sessao
GET	/ /dashboard	Pagina principal do jogo
GET	/api/state	Estado completo do jogo (JSON)
POST	/api/build/<slot_id>	Iniciar construcao
POST	/api/task/<slot_id>/start	Iniciar tarefa
POST	/api/task/<slot_id>/collect	Recolher recompensa
POST	/api/chop	Cortar arvore
POST	/api/mine-stone	Minerar pedra
POST	/api/slot/<slot_id>/remove	Remover construcao
POST	/api/inventory/remove	Remover recursos manualmente
GET	/leaderboard	Pagina de classificacao
GET	/api/leaderboard	Top 10 jogadores (JSON)
GET	/skins	Loja de skins
GET	/api/skins	Lista de skins (JSON)
POST	/api/skin/select	Selecionar skin

8. Instalacao e Execucao

Requisitos minimos: Python 3.12+, pip, navegador atual.

```
# 1. Clonar repositorio e entrar na pasta
git clone <url> && cd minecraft2d

# 2. Criar e ativar ambiente virtual
python3 -m venv .venv
source .venv/bin/activate

# 3. Instalar dependencias
pip install -r requirements.txt

# 4. Executar o servidor
python server.py

# 5. Abrir no navegador
http://127.0.0.1:8001
```


9. Conclusão

O projeto Minecraft 2D foi desenvolvido com sucesso, cumprindo todos os requisitos obrigatórios definidos no enunciado e diversas funcionalidades extra:

- Sistema de autenticação completo (registro, login, logout) com sessões persistidas
- 4 tipos de recursos (madeira, pedra, ferro, diamantes) com limites
- 3 tipos de construções distintas com custos, tempos e funcionalidades
- 4 slots de construção por jogador com máquina de estados (5 estados)
- Mecânica de tempo sem cron jobs (comparação de timestamps)
- Sistema de tarefas com recompensas recolhidas manualmente
- Frontend responsivo (desktop, tablet, mobile)
- Comunicação frontend-backend via Fetch API (JSON)
- Leaderboard global (top 10) com atualização em tempo real
- Loja de skins com 5 opções e filtros CSS
- Sistema de evolução do machado com 4+ níveis
- Modo escuro com transição dia/noite e efeitos visuais
- Base de dados SQLite com esquema completo e dados iniciais (seed)
- Código modular com Blueprints e separação MVC

Para além dos requisitos mínimos, o projeto inclui skins com filtros CSS, sistema de machado com upgrades progressivos, cena 2D interativa com animações, tema claro/escuro com efeitos visuais (nuvens, estrelas, pirilampos), e uma interface polida e consistente com o tema Minecraft.

O código fonte encontra-se organizado de forma modular, com separação clara entre modelos, rotas e templates, facilitando a manutenção e evolução futura do projeto.

10. Referências

- Flask Documentation (2025). <https://flask.palletsprojects.com/>
- MDN Web Docs (2025). Fetch API. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- MDN Web Docs (2025). HTML. <https://developer.mozilla.org/en-US/docs/Web/HTML>
- Flask-Login Documentation. <https://flask-login.readthedocs.io/>
- passlib Documentation. <https://passlib.readthedocs.io/>
- OWASP (2025). Top 10 Web Application Security Risks. <https://owasp.org/www-project-top-ten/>